

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

APPENDIX D

ONE-TO-MANY | SERVERPOD

 Copy as Markdown

One-to-many

One-to-many (1:n) relationships describe a scenario where multiple records from one table can relate to a single record in another table. An example would be the relationship between a company and its employees, where multiple employees can be employed at a single company.

You can define the relation from the 'many' side (like `Employee`), from the 'one' side (like `Company`), or on both. Defining it from the 'one' side hides the foreign key from your class, which keeps the model minimal; defining it from the 'many' side gives you the foreign key as a field you can read and filter on directly.

Defining the relationship

In the following examples we show how to configure a 1:n relationship between `Company` and `Employee`.

Implicit definition

With an implicit setup, Serverpod determines and establishes the relationship based on the table and class structures.

```
# company.spy.yaml
class: Company
table: company
fields:
  name: String
  employees: List<Employee>?, relation

# employee.spy.yaml
class: Employee
table: employee
fields:
  name: String
```



Ask AI

In the example, we define a 1:n relation between `Company` and `Employee` by using the `List<Employee>` type on the `employees` field together with the `relation` keyword.

Serverpod adds the corresponding foreign key column to the 'many' side's table (e.g., `employee`), but it does not appear as a field on the class.

When fetching companies, you can now include any or all employees in the query. 1:n relations also enable additional [filtering](#) and [sorting](#) operations for [relational queries](#).

Explicit definition

In an explicit definition, you directly specify the relationship in a one-to-many relation.

This can be done through an [object relation](#):

```
# company.spy.yaml
class: Company
table: company
fields:
  name: String

# employee.spy.yaml
class: Employee
table: employee
fields:
  name: String
  company: Company?, relation
```

Or through a [foreign key field](#):

```
# company.spy.yaml
class: Company
table: company
fields:
  name: String

# employee.spy.yaml
class: Employee
table: employee
fields:
  name: String
  companyId: int, relation(parent=company)
```

The examples are 1:n relations because there is **no** unique index constraint on the foreign key field. This means that multiple employees can reference the same company.

Bidirectional relations

For a more comprehensive representation, you can define the relationship from both sides.

Either through an [object relation](#) on the many side:

```
# company.spy.yaml
class: Company
table: company
fields:
  name: String
  employees: List<Employee>?, relation(name=company_employees)

# employee.spy.yaml
class: Employee
table: employee
fields:
  name: String
  company: Company?, relation(name=company_employees)
```

Or through a [foreign key field](#) on the many side:

```
# company.spy.yaml
class: Company
table: company
fields:
  name: String
  employees: List<Employee>?, relation(name=company_employees)

# employee.spy.yaml
class: Employee
table: employee
fields:
  name: String
  companyId: int, relation(name=company_employees, parent=company)
```

Just as in the 1:1 examples, the `name` parameter takes a unique string that links both sides together.